OpsCrew: technical overview

HackUTD 2026 · CBRE · NVIDIA Nemotron · NMC² · repo `anothergrind/hackutd-2026`, branch `opscrew-agents` · Nov 7, 2026

What it is. OpsCrew is a team of five AI agents that catches building failures before they happen. A signal (a ticket, sensor alarm, or the nightly sweep) starts a run. A Lead agent reads the building's memory and sends three specialists out in parallel: Ticket Triage links small tickets into one problem, Sensor Watch fits a trend and estimates hours until a limit, and Lease / SLA finds the tenants hurt and their penalty per hour. The Lead prices the cost of waiting against fixing now with deterministic math, then has Dispatch draft the fix: a work order, a temporary schedule change and tenant notices. Every side effect waits in an approval queue until a manager approves it; approving executes it.

Contents.

1. Tracks and how they map
2. Tech stack
3. Architecture
4. End-to-end run workflow
5. Agent runtime internals
6. Agents and all 19 tools
7. Deterministic math
8. Approval gate
9. Nightly sweep
10. Data model
11. Demo scenarios
12. API reference
13. Frontend
14. Modes, config and fallbacks
15. Testing
16. Safety, limits and known gaps
17. File map

1. Tracks and how they map

Track	Requirement (from the opening slides)	Where OpsCrew meets it
CBRE	Connect fragmented real estate data to predict operational failures, quantify business impact, or execute solutions before disruptions spread. A working prototype, not passive visualization.	**Connect:** tickets, telemetry, asset dependency graph, leases/SLAs, weather, vendors and parts each feed a different agent. **Predict:** ticket linking + trend to limit. **Quantify:** `price_risk` ($ wait vs $ fix). **Execute:** approval inserts real work orders, notices and schedule changes.
NVIDIA	Use Nemotron beyond a chatbot: multi-step workflows (agents that plan and execute), tool integration (external APIs), real-world applicability.	Runs start from events or the sweep, no prompt. Lead plans, delegates via `ask_*` tools, specialists run concurrently, 19 tools incl. the live Open-Meteo API, human gate, full audit log.
NMC²	Help data center teams stay ahead of missed warnings, unclear handoffs and wrong information.	Demo site is a data center (Avon DC1). Sweep catches unacted warnings; approval queue is the handoff; building memory carries context between runs.
General	Design, Virality, Beginner, Hardware; opt in on Devpost (not stated as automatic).	Design: HackTX NMC³ design system, light/dark. Others depend on team and video.

2. Tech stack

Layer	Technology	How it is used
Web framework	Next.js 16.4 (App Router, Turbopack), React 19, TypeScript (strict)	One app serves pages and API route handlers. `after()` from `next/server` keeps the agent run going after the HTTP response returns.
Database	Supabase (Postgres + PostgREST) via `@supabase/supabase-js`	All state. Server uses the service-role client (bypasses RLS); tables have public-read RLS policies. Falls back to an in-memory database when keys are missing (section 14).

LLM	NVIDIA NIM, OpenAI-compatible `/v1/chat/completions`; default `nvidia/llama-3.3-nemotron-super-49b-v1.5`	Plain `fetch`, no SDK. Function calling with `tool_choice: "auto"`, `temperature 0.2`, `max_tokens 2048`, 60 s timeout, 3 attempts with backoff on 429/5xx.
External data API	Open-Meteo forecast API (free, no key)	`get_weather` tool and the sweep's forecast check.
Scheduling	Vercel Cron (`vercel.json`)	`GET /api/sweep/cron` daily at 07:00 UTC (2 AM CT), authorized by `CRON_SECRET`.
UI	CSS modules + design tokens ported from HackTX NMC³; IBM Plex Sans/Mono via `next/font`; `lucide-react` icons	Light/dark themes, animated running borders, flowing edge dashes, polling for live updates.
Tooling	`node:test`, `tsx`, `tsc`	Unit tests for graph and math, end-to-end dry-run script, typecheck.

3. Architecture

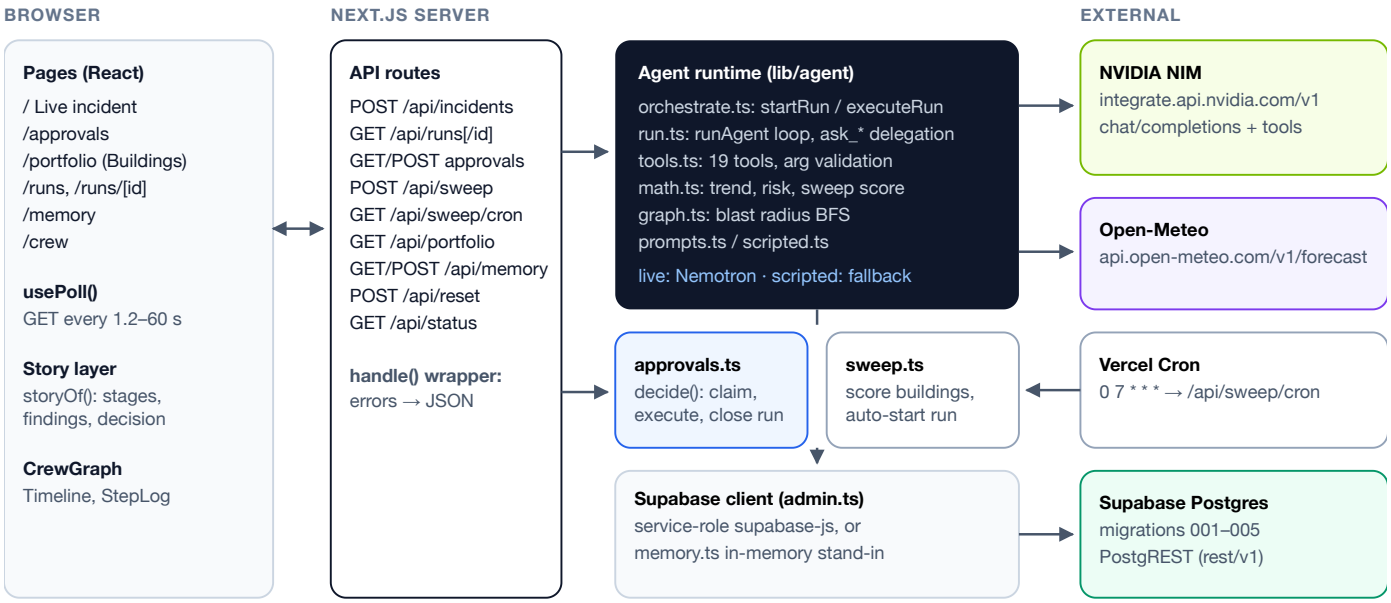


Figure 1. Everything runs in one Next.js app. The browser polls API routes; route handlers call the agent runtime, which calls Nemotron, Open-Meteo and the database.

4. End-to-end run workflow

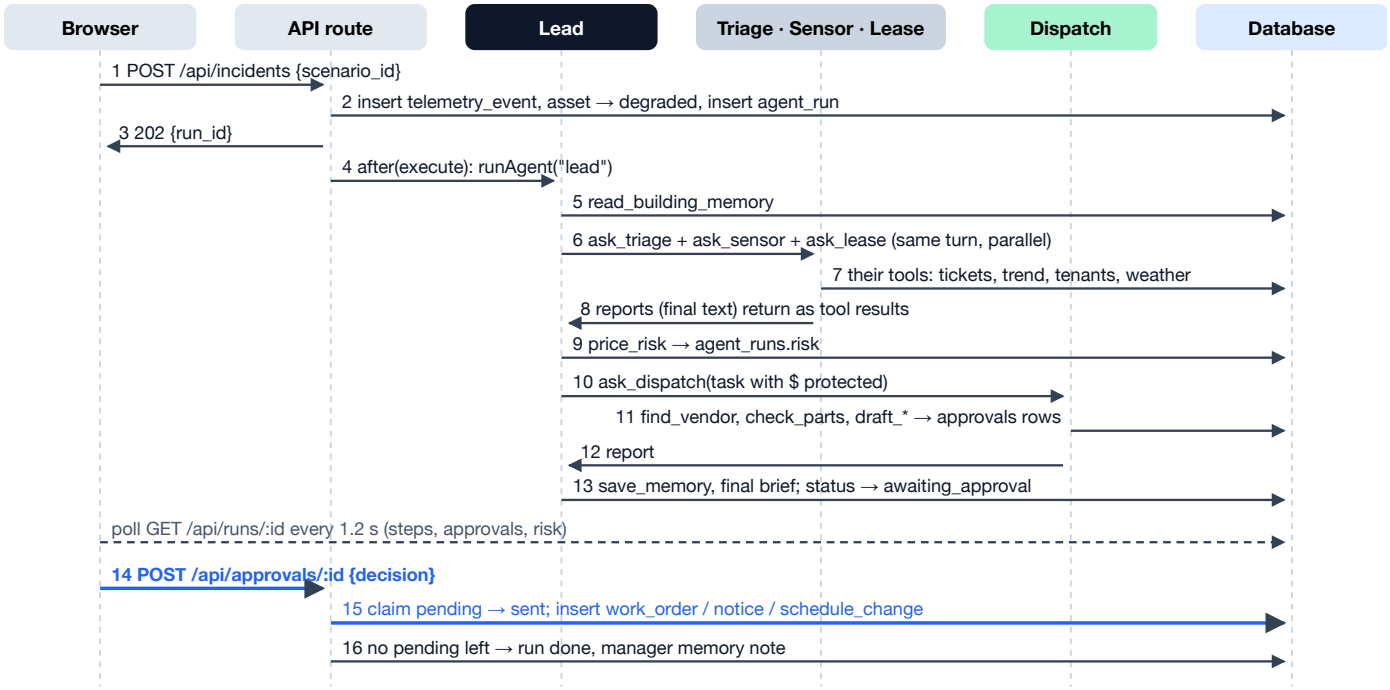


Figure 2. Sequence of one incident run. Every agent step is also written to `agent_steps` as it happens, which is what the UI draws.

Step by step

1. **Trigger.** The UI calls `POST /api/incidents {scenario_id}` (or the sweep starts a run, section 9). The route looks up the scenario's signal and hints.
2. **startRun** (`orchestrate.ts`): inserts the signal into `telemetry_events`, sets that asset's `status` to `degraded`, inserts `agent_runs` (`status running`, `trigger`, `title`, `property_id`, `mode` live|scripted). Returns `{runId, execute}`.
3. The route returns **202 {run_id}** immediately and schedules `execute` with `after()` (`maxDuration = 300` s).
4. **executeRun** builds the run context (`RunCtx`: db, runId, propertyId, signal, hints, an empty `facts` bag, and `log()` which inserts into `agent_steps`). It loads the property name and composes the Lead's task: the signal JSON plus site context (likely root asset, impacted asset, asset to price, expected outage, metric to watch).
5. **Lead loop** runs (section 5). Typical order: `read_building_memory` → `ask_triage`, `ask_sensor`, `ask_lease` in one turn (executed with `Promise.all`, so the three specialists run concurrently) → `price_risk` → `ask_dispatch` → `save_memory` → final brief.
6. Each `ask_*` call recursively runs that specialist's own loop with its own tools and system prompt; its final text becomes the tool result `{report}` the Lead sees.
7. When the Lead finishes, the run is set to `awaiting_approval` if any approval is pending, otherwise `done` (with `finished_at`). Any thrown error sets `error` and logs an error step.
8. The browser polls `GET /api/runs/:id` every 1.2 s and redraws the stages, findings, decision panel, crew graph and timeline from the steps.
9. The manager approves (one by one, with optional edits, or "Approve all"). Each approval executes its side effect (section 8). When no approvals remain pending, the run becomes `done` and a manager memory note is saved.

5. Agent runtime internals (`run.ts`)

Aspect	Behavior
Mode selection	`agentMode()` = `scripted` if `OPSCREW_MODE=scripted` or no `NVIDIA_API_KEY`, else `live`. Stored on the run.
Turn limits	Lead 10, Triage 5, Sensor 5, Lease 5, Dispatch 6. Hitting the limit ends the agent with "Stopped after the turn limit".

Live loop	Messages = [system prompt, task]. Each turn: `chat(messages, toolSpecs)`. No tool calls → final answer, logged as `final`. Text alongside tool calls → logged as `thought`. Tool call arguments are JSON-parsed (unparseable → `{_unparsed}`).
Parallelism	All tool calls in one assistant turn run with `Promise.all`; results are appended in call order as `role: tool` messages (truncated to 6,000 chars each).
Delegation	The Lead's `ask_triage / ask_sensor / ask_lease / ask_dispatch` are tools whose `run()` calls `runAgent(specialist, task)` and returns `{report}`.
Tool errors	A tool that throws returns `{error: message}` to the model instead of crashing the run, so the model can retry with valid ids. Every call is logged as a `tool` step with `{name, args, result, ms}`.
Nemotron client	`nemotron.ts`: up to 3 attempts, backoff 0.8 s / 1.6 s on 429 or 5xx, no retry on other 4xx, 60 s request timeout. Strips `<think>…</think>` reasoning from content.
Fallback	If a live agent's Nemotron call fails and that agent has not yet called a write tool (any `draft_*`, `save_memory`, or `ask_dispatch`), it logs an error step and continues with the scripted policy. Disable with `OPSCREW_FALLBACK=false`. After a write, the error propagates.
Scripted policy	`scripted.ts`: per-agent, per-turn moves that call the same tools against the same data, deciding arguments from the run's `facts` (filled in by tool results) and the scenario `hints`. Pauses `OPSCREW_PACE_MS` (default 700 ms) between turns so the live view is followable.
Step kinds	`start` (task), `thought`, `tool`, `final`, `error`; each row has `agent` and `created_at`.

System prompts (`prompts.ts`), condensed

- **Lead:** read memory; call the three specialists in one turn; `price_risk` on the asset whose failure hurts tenants most; if waiting costs more, `ask_dispatch` with the dollar amount; `save_memory`; finish with a 3–4 sentence brief and state that nothing is sent without approval.
- **Triage:** `search_tickets` then `link_tickets` for shared root cause; report count, span, problem, root asset.
- **Sensor:** `get_dependencies` then `detect_trend` (or `get_readings`); report slope, value, hours to limit, downstream.
- **Lease:** `tenants_at_risk` then `get_weather`; report tenants, $/hr, exposure, weather.
- **Dispatch:** `find_vendor`, `check_parts`, then `draft_work_order`, optional `draft_schedule_change`, one `draft_tenant_notice` per tenant.
- Shared rules: only use ids returned by tools; never invent numbers; reply with plain text and no tool call when done.

6. Agents and all 19 tools

Each agent sees only its own tools (least privilege). "Writes" tools only create pending approvals (or a memory note); none sends anything.

Agent	Tool	Arguments (validated)	What it does / reads / writes
Lead	`read_building_memory`	`asset_id?`	Reads up to 8 newest `building_memory` notes for the run's property (optionally one asset).
	`ask_triage`	`task` (≤800 chars)	Runs Ticket Triage; returns its report.
	`ask_sensor`	`task`	Runs Sensor Watch; returns its report.
	`ask_lease`	`task`	Runs Lease / SLA; returns its report.
	`price_risk`	`asset_id`, `outage_hours` 0.5–72, `repair_cost_usd` 0–1M, `hours_until_failure?`	Sums SLA penalties of tenants on the asset and everything downstream (blast radius), runs `priceRisk()`, saves the result to `agent_runs.risk`.
	`ask_dispatch`	`task`	Runs Dispatch; returns its report.
	`save_memory` `writes`	`note` ≤600, `asset_id?`	Inserts a `building_memory` row (source agent, run_id).
Triage	`search_tickets`	`days` 1–365 (default 30), `equipment[]?`	Open (not resolved) tickets for the property since N days, newest first, max 25; notes truncated to 200 chars.
	`link_tickets`	`ticket_ids[]`, `problem`, `root_asset_id`	Verifies every ticket id and the asset exist, records the link in run facts. Returns linked titles.
Sensor	`get_dependencies`	`asset_id`	Upstream ancestors (reverse BFS) and downstream blast radius with hop counts and relations.
	`get_readings`	`asset_id`, `metric?`, `days` 1–90 (default 14)	Up to 60 telemetry rows, oldest first.
	`detect_trend`	`asset_id`, `metric`, `limit`	Least-squares line over up to 200 numeric readings (needs ≥3). Returns slope/day, latest, r^2, hours to limit.
Lease	`tenants_at_risk`	`asset_id`, `outage_hours` 0.5–72	Blast radius → tenants on any reached asset; per-tenant exposure = penalty/hr × hours; total penalty/hr.
	`get_weather`	`asset_id`	Open-Meteo for the property's lat/lon: current temp/wind/precip and 2-day max temp, rain probability, gusts (°F, mph, America/Chicago). 6 s timeout.
Dispatch	`find_vendor`	`asset_id`, `est_hours` (default scenario repair hours or 4)	Vendors whose `certified_kinds` contain the asset kind, soonest `next_available` first, max 5; est cost = callout + rate × hours.
	`check_parts`	`asset_id`	Parts for the asset: on hand, unit cost, lead days.
	`draft_work_order` `writes`	`asset_id`, `vendor_id`, `priority` 1–5, `title`, `steps[]`, `est_hours?`, `protects_usd?`	Pending approval, payload {asset, vendor, priority, steps, est_hours, scheduled_for = vendor's next slot}, cost computed from the vendor.
	`draft_schedule_change` `writes`	`asset_id`, `change`, `hours` 0.5–168, `protects_usd?`	Pending approval for a temporary change (pre-cool, shift load, pre-start generator).
	`draft_tenant_notice` `writes`	`tenant_id`, `subject`, `message` ≤2000, `protects_usd?`	Verifies tenant, pending approval with recipient email, subject, message.

Validation helpers reject empty strings, out-of-range numbers and unknown ids with a message telling the model what to use instead. Strings are trimmed and length-capped; arrays are filtered to strings and capped.

7. Deterministic math (`math.ts`, `graph.ts`)

The model decides *what* to ask; code decides the numbers, so every figure on screen is reproducible.

Trend and time to limit

```
x = hours since epoch, y = reading     (needs n ≥ 3)
slope = Σ(x−x̄)(y−ȳ) / Σ(x−x̄)²         per hour;  per day = slope × 24
r²    = (Σ(x−x̄)(y−ȳ))² / (Σ(x−x̄)² · Σ(y−ȳ)²)
hours_to_limit = (limit − latest) / slope   (null if slope is 0 or moving away from the limit)
```

Risk price

```
penalties      = Σ tenant SLA $/hr (asset + downstream) × outage_hours
emergency      = repair_cost × 2.5
wait_usd       = penalties + emergency          fix_usd = repair_cost
saved_usd      = wait_usd − fix_usd
urgency        = "act today" (≤24 h) · "act this week" (≤72 h) · "plan" (>72 h) · "watch" (unknown)
```

Demo example (Cooling cascade): CHILLER-1 downstream tenants Lonestar Cloud $25,000/hr + MedRecords $12,000/hr = $37,000/hr × 3 h = $111,000; repair $1,010 × 2.5 = $2,525; wait $113,525 vs fix $1,010; trend 0.406 °F/day, latest 49.69 °F, limit 50 °F → 18.3 h (r^2 0.997) → "act today".

Blast radius

Breadth-first search over `dependencies` (upstream → downstream: powers, cools, controls, serves), cycle-safe, records hop depth and the edge that reached each asset. `upstreamOf` runs the same BFS on reversed edges.

Sweep score (0–100)

Signal	Points	Cap
Open tickets	urgency × 3 each	30
Sensor anomalies in last 72 h (non-ticket, excluding the trend metric)	8 each	20
Degraded / failed assets	10 each	20
Trend crosses limit	≤24 h: 30 · ≤72 h: 18 · later: 6	30
2-day forecast high	≥95 °F: 10 · ≥88 °F: 5	10

8. Approval gate (`approvals.ts`)

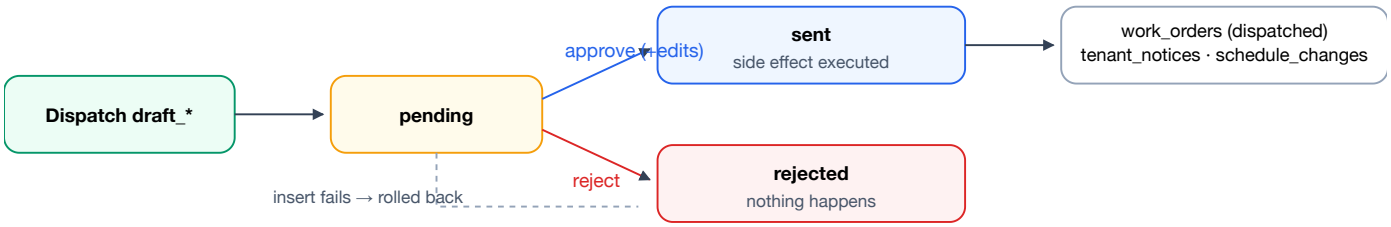


Figure 3. Approval states. Only `pending` can change; the change is a conditional update, so concurrent clicks cannot execute twice.

1. Load the approval; refuse if not `pending` (409 "Already sent/rejected").
2. Apply only whitelisted edits: work order `title, steps, priority`; tenant notice `subject, message`; schedule change `change, hours`. Values are type-checked and length-capped.
3. **Claim:** `UPDATE approvals SET status=sent|rejected … WHERE id=? AND status='pending'`. Zero rows → someone else decided first (409).
4. **Execute** on approve: insert into `work_orders` (status dispatched, vendor, scheduled_for), `tenant_notices` (subject, message), or `schedule_changes` (ends_at = now + hours). On failure the claim is rolled back to pending.
5. **Close:** if no approvals for the run remain pending and the run is `awaiting_approval`, set `done` + `finished_at` and write a manager memory note ("approved N of M drafted actions").

Run status machine: `running` → `awaiting_approval` → `done`, or `running` → `done` (nothing drafted), or any → `error`.

9. Nightly sweep (`sweep.ts`)

1. Insert a `sweeps` row (running).
2. Load all properties, open tickets, 30 days of telemetry, and assets in parallel.
3. Per property: count anomalies, degraded assets, tickets; if the property's scenario defines a trend metric, fit it and compute hours to limit; fetch the 2-day forecast max (5 s timeout, failures ignored). Score with `sweepScore()`, keeping each reason and its points.
4. Sort by score. If `autostart` and the top score ≥ **40** and that property has no run currently running, start a run with `trigger: sweep` using that property's scenario hints; the signal message lists the reasons.
5. Save scores and the started run id; the route hands the run's `execute` to `after()`.

Triggered by the "Nightly sweep" button (`POST /api/sweep`), "Score buildings" on the Buildings page (`autostart:false`), or Vercel Cron (`GET /api/sweep/cron` with `Authorization: Bearer $CRON_SECRET`). Demo result: Avon DC1 81, Avon Center 21, Northstar Tower 6 → DC1 auto-started.

10. Data model (Supabase migrations 001–005)

Table	From	Key columns and purpose
`properties`	003/004	id, name, address, building_type, lat, lon. Three demo buildings: Avon Center (mixed-use), Northstar Tower (office), Avon DC1 (data center).
`assets`	004	id = equipment tag (e.g. CHILLER-1), property_id, name, kind (power, cooling, hvac, controls, space), location, status (ok, degraded, failed).
`dependencies`	004	upstream, downstream, relation (powers, cools, controls, serves). DC1: utility/generator → switchgear → UPS → halls; BMS controls chiller + CRAHs; chiller → loop → CRAHs → halls.
`tenants`	004	property_id, asset_id (space), name, sla_penalty_per_hour, contact_email. Lonestar Cloud $25k/hr, MedRecords $12k/hr, Parkway Capital $1.5k/hr, Brightline Legal $800/hr.
`maintenance_tickets`	003/005	property_id, equipment (joins assets.id), title, notes, status, urgency 1–5, reported_at. Seed adds 3 small DC1 tickets over 13 days.
`telemetry_events`	004/005	asset_id, source (sensor, bms, cyber, ticket, weather), metric, value, message, created_at. Seed adds 15 days of CHILLER-1 supply_temp_f rising ~0.41 °F/day, plus pump amps, inlet temp, vibration, UPS health. Signals are also stored here.
`energy_readings`, `incidents`	003	Starter tables, not used by the agents.
`agent_runs`	004/005	event_id, status, trigger, title, property_id, risk (jsonb), mode, summary, created_at, finished_at.
`agent_steps`	004/005	run_id, agent, kind (start, thought, tool, final, error), content (jsonb), created_at. The audit trail and the UI's source.
`approvals`	005	run_id, kind (work_order, tenant_notice, schedule_change), title, payload (jsonb), protects_usd, cost_usd, status (pending, sent, rejected), decided_by, decision_note, decided_at.
`work_orders`	004/005	run_id, approval_id, asset_id, vendor_id, priority, title, steps[], status, scheduled_for.
`tenant_notices`	004/005	run_id, approval_id, tenant_id, subject, message.
`schedule_changes`	005	approval_id, run_id, asset_id, change, starts_at, ends_at.
`vendors`	005	name, trade, certified_kinds[], response_hours, hourly_rate, callout_fee, next_available, rating, phone. 5 seeded.
`parts`	005	asset_id, part, on_hand, unit_cost, lead_days.
`building_memory`	005	property_id, asset_id, note, source (agent, manager, seed), run_id.
`sweeps`	005	status, scores (jsonb with reasons), started_run_id, finished_at.
`projects`, `tracks`	001/002	Starter tables (health check, hackathon tracks), not used by OpsCrew.

RLS is enabled on every table with a public-read policy; all writes go through the server's service-role key.

11. Demo scenarios (`scenarios.ts`)

Scenario	Signal	Hints (root / impacted / priced asset, outage, trend)	Demo result
Cooling cascade (default)	CRAH-A1 ticket: third report of water, musty smell, warm hot aisle	CRAH-A1 / HALL-A / CHILLER-1, 3 h outage, trend CHILLER-1 supply_temp_f limit 50, repair 4 h, schedule change pre-cool HALL-A	wait $113,525 vs fix $1,010; 4 drafts
BMS intrusion	BMS-1 cyber: 14 unauthorized setpoint writes	BMS-1 / HALL-A / BMS-1, 2 h, isolate + restore setpoints, manual control	wait $76,113 vs $845; 4 drafts
Utility sag + storms	UTIL-FEED-1 voltage 82%	GEN-1 / HALL-A / SWGR-1, 4 h, generator test, UPS modules, pre-start generator	wait $150,113 vs $845

Office AHU leak	AHU-04 ticket: moisture under vent	AHU-04 / FLOOR-4-EAST / AHU-04, 8 h, drain pan (3-day lead)	wait $8,513 vs $845; 2 drafts

Hints steer the scripted policy and are given to Nemotron as "site team context (may be incomplete)"; the live model can still choose differently.

12. API reference

Endpoint	Input → output
`GET /api/incidents`	Scenario list (without hints).
`POST /api/incidents`	`{scenario_id}` → 202 `{run_id}`; run continues in `after()`. `/api/simulate` is an alias.
`GET /api/runs`	40 newest runs with property name and pending-approval count.
`GET /api/runs/:id`	`{run (+property, signal), steps[], approvals[]}`; polled by the UI.
`GET /api/approvals?status=`	`pending` (default) or `all`; 100 newest with run title and property.
`POST /api/approvals/:id`	`{decision: approve|reject, edits?, by?, note?}` → `{id, status}`; 409 if already decided.
`GET /api/sweep` / `POST /api/sweep`	Latest sweep / run one: `{autostart?}` → `{sweep_id, scores, run_id}`.
`GET /api/sweep/cron`	Vercel Cron; requires `Authorization: Bearer $CRON_SECRET`.
`GET /api/portfolio`	Properties, assets, dependencies, tenants, open tickets, 20 recent work orders, latest sweep.
`GET/POST /api/memory`	List 100 notes / add a manager note `{property_id, asset_id?, note}`.
`POST /api/reset`	Deletes runs, approvals, executed actions, sweeps, non-seed memory and injected signals; assets back to ok. Requires `ALLOW_DEMO_RESET=true` unless on the in-memory database.
`GET /api/status`	`{mode, model, database: memory|supabase, reset_enabled}`.

All handlers are wrapped in `handle()`: thrown errors become JSON `{error}` (503 for missing Supabase config, else 500); bad input returns 400.

13. Frontend

Pages

Page	Contents
`/` Live incident	Hero (what OpsCrew is), "Run the demo incident", other scenarios, Nightly sweep, Reset. Then for the current run: incident line, 5-stage stepper, "What the crew found" (4 finding cards + Lead summary), "Your decision" (wait vs fix, drafted actions, Approve all), "How the agents worked it out" (crew graph, timeline, step log).
`/approvals`	Pending approvals (edit, approve, reject) and recently decided.
`/portfolio` Buildings	Last sweep scores with reasons; per building, assets grouped power → controls → cooling → hvac → space with status dots, tenants and $/hr; work orders sent.
`/runs`, `/runs/:id`	History table (status, $ at risk, saved) and full replay.
`/memory`	All notes by source; form to add a manager note.
`/crew`	Each agent, what data it owns, its tools and which ones write.

How the UI turns steps into a story (`lib/view.ts`)

- **deriveAgents:** per agent, state PENDING → RUNNING (start step) → SUCCEEDED (final) or FAILED (run error); latest thought or tool summary as the "now" line; tool steps for chips.
- **summarize:** one-line result per tool (e.g. `detect_trend` → "+0.406/day · 18 h to limit").
- **storyOf:** five stages (Signal, Investigate, Price it, You approve, Fixed early), each idle / active / waiting / done with a live sentence; four findings (history, tickets, sensors, tenants) from the latest result of the relevant tool.

Components and live updates

- **CrewGraph:** fixed layout (Trigger → Lead → 4 specialists → latest 2 tool chips each → Approval queue), bezier edges with a moving dash on active work, animated conic border on running nodes, scaled to fit with `ResizeObserver`.
- **Timeline:** one lane per agent from its start to final step; overlapping bars show parallel work; dots per step.

- **StepLog:** every step newest first; tool rows expand to exact args and result JSON.
- **Decision / ApprovalCard:** money panel with formula; each draft expands to an editable card.
- Polling (`usePoll`): run detail 1.2 s; approvals page 3 s; sidebar badge and runs 4 s; memory 6 s; buildings 8 s; status 60 s.
- Design: HackTX NMC³ tokens (neutral surfaces, hairline borders, one blue accent, green done, orange waiting, red failed), per-agent colors, light/dark via `data-theme` with a circular view transition, film-grain overlay, reduced-motion support.

14. Modes, configuration and fallbacks

Variable	Effect
`NEXT_PUBLIC_SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`	Real database. If either is missing, the in-memory database is used.
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Used by the starter routes (health, projects, tracks).
`NVIDIA_API_KEY`	Live Nemotron. Missing → scripted mode.
`NEMOTRON_MODEL`, `NVIDIA_BASE_URL`	Model id (check with `npm run smoke`) and endpoint override.
`OPSCREW_MODE=scripted`	Force scripted even with a key (stage fallback).
`OPSCREW_FALLBACK=false`	Disable per-agent fallback to scripted on Nemotron errors.
`OPSCREW_PACE_MS`	Pause between scripted turns (default 700; 0 in the dry run).
`ALLOW_DEMO_RESET=true`	Enable reset against Supabase.
`CRON_SECRET`	Required by the cron route.

In-memory database (`lib/supabase/memory.ts`): a stand-in for the Supabase client seeded like migrations 003–005 (3 properties, 17 assets, 19 dependencies, 4 tenants, 6 tickets, 19 telemetry rows, 5 vendors, 5 parts, 3 memory notes). Implements only the query-builder calls the app uses: select (with nested embeds such as `agent_runs(title, properties(name))`), insert/update/delete with `.select()`, `eq, neq, in, gte, contains, not`, `order, limit, single`, and `count: exact, head`. State is kept on `globalThis` so all route handlers share it; it resets when the server restarts. The sidebar shows "Demo mode" and the status route reports `database: memory`.

15. Testing

Command	What it checks
`npm test`	7 unit tests (`node:test`): blast radius incl. cycles and leaf; trend slope and r^2; ≥3-point rule; hours-to-limit rising/falling/away; risk price arithmetic; sweep score cap and zero case.
`npm run dry-run [scenario]`	Runs the real orchestrator, tools, approval executor and sweep against the in-memory database; prints every step, the risk and brief; approves all drafts (one with an edit); confirms double-approve is blocked; runs the sweep and its auto-started run.
`npm run smoke`	Lists Nemotron models for your key and checks a tool-calling round trip.
`npm run typecheck`, `next build`	Types and production build.

16. Safety, limits and known gaps

Safeguards

- Least-privilege tools per agent; only draft tools can lead to side effects, and only through a human approval.
- All LLM tool arguments are validated; unknown ids are rejected with guidance.
- Numbers come from deterministic code, not model text.
- Approve is claim-first and idempotent; failed executions roll back.
- Service-role key is server-only; reset is guarded; cron requires a secret.
- Every step is logged with arguments, results and timing.

Known gaps (say these honestly if asked)

- All building data is synthetic seed data; the only live external data is the Open-Meteo forecast. No real CMMS/BMS/lease system is connected.
- Weather is shown and adds sweep points but does not change the predicted failure time or the dollar figures.
- Outage hours and the 2.5× emergency premium are per-scenario assumptions, not data-driven.
- The trend is a straight line on clean data; real sensors are noisy and failures can be sudden.
- In live mode, the Lead's prose brief is not cross-checked against tool results.
- Agents do not search past runs or work orders for repeat failures (memory notes only).
- Updates use polling, not realtime subscriptions. No user authentication; `/api/incidents` and `/api/sweep` spend NVIDIA credits if deployed publicly.
- Live Nemotron and a real Supabase project have not yet been exercised end to end in this environment (no keys were configured); everything else was verified with the dry run and screenshots.

Next improvements

1. Verification step comparing the brief's numbers to tool results.
2. Past-incident lookup tool on the same asset.
3. Weather-adjusted time to failure and storm-adjusted outage hours.
4. National Weather Service alerts and ERCOT grid conditions as live data.

---

17. File map

Path	Role
`src/lib/roster.ts`	Agents, colors, tools (shared by UI and server)
`src/lib/scenarios.ts`	Demo incidents and hints
`src/lib/types.ts`	Run, Step, Approval, Risk, Sweep shapes
`src/lib/nemotron.ts`	NIM client, retries, mode selection
`src/lib/agent/run.ts`	Agent loop, delegation, parallel calls, fallback
`src/lib/agent/tools.ts`	All tools and validation
`src/lib/agent/prompts.ts`, `scripted.ts`	Live system prompts; scripted policy
`src/lib/agent/orchestrate.ts`	startRun / executeRun
`src/lib/agent/math.ts`, `graph.ts` (+ tests)	Trend, risk, sweep score, BFS
`src/lib/approvals.ts`, `src/lib/sweep.ts`	Approval executor; nightly sweep
`src/lib/supabase/admin.ts`, `memory.ts`	DB client selection; in-memory stand-in
`src/lib/view.ts`, `client.ts`, `http.ts`	Story derivation; polling and formatting; route error wrapper
`src/app/**`	Pages and API routes
`src/components/{run,story,approvals,shell,ui}`	Crew graph, timeline, step log, stepper, findings, decision, approval card, sidebar, icons
`supabase/migrations/001–005`	Schema and seed data
`scripts/dry-run.ts`, `nemotron-smoke.mjs`	End-to-end dry run; Nemotron check
`vercel.json`, `next.config.ts`, `.env.example`	Cron schedule, dev indicator, configuration